UNIVERSIDAD CATOLICA DEL MAULE

Computación de Alto Rendimiento

Javier E. Rojas Núñez

jerojas@ucm.cl

UNIVERSIDAD CATOLICA DEL MAULE



Pools de procesos

Listas de tareas

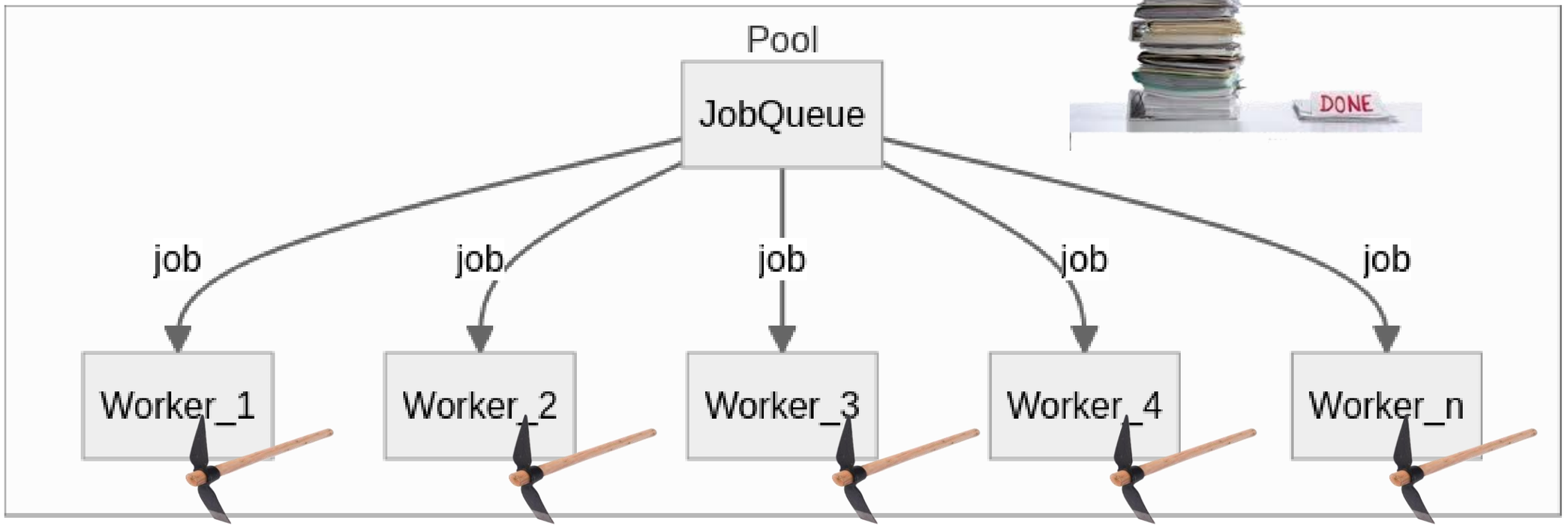
Cuando hay una gran cantidad de trabajos a distribuir entre procesos se puede sobrepasar (y por mucho) el número de procesadores de nuestra máquina.



- Mandar muchos procesos a la máquina jamás será recomendable
- Es necesario no solo dividir la tarea en las partes individuales, también es necesario asignar la carga a los procesadores
- Esto es fácil cuando el número de tareas y procesadores es conocido, pero no así para valores arbitrarios

El pool de procesos

Para evitar tener que pensar en como distribuir todas las tareas generadas podemos crear una piscina de tareas o pool.





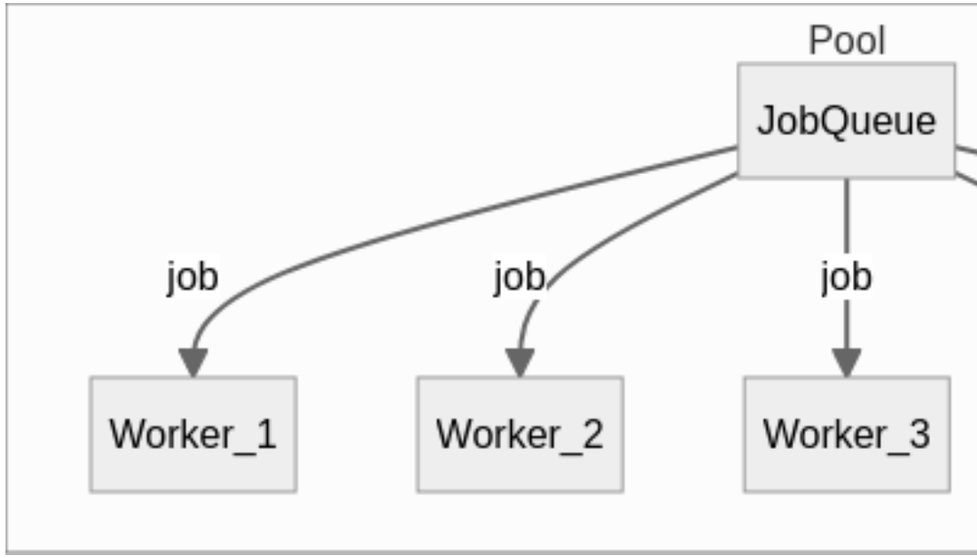
Creando un pool

Al crear un pool se crea la estructura para distribuir la carga de trabajo entre una cantidad determinada de procesos

Pool([processes[, initializer[, initargs[, maxtasksperchild]]]]

- Processes: Cantidad de procesos
- Initializer/ initargs: Un objeto *callable* que se ejecuta en cada proceso

  Initializer(*initargs)
- Maxtaskperchild: Cantidad de tareas que se puede ejecutar por proceso antes de cerrarse y ser reemplazado por un nuevo



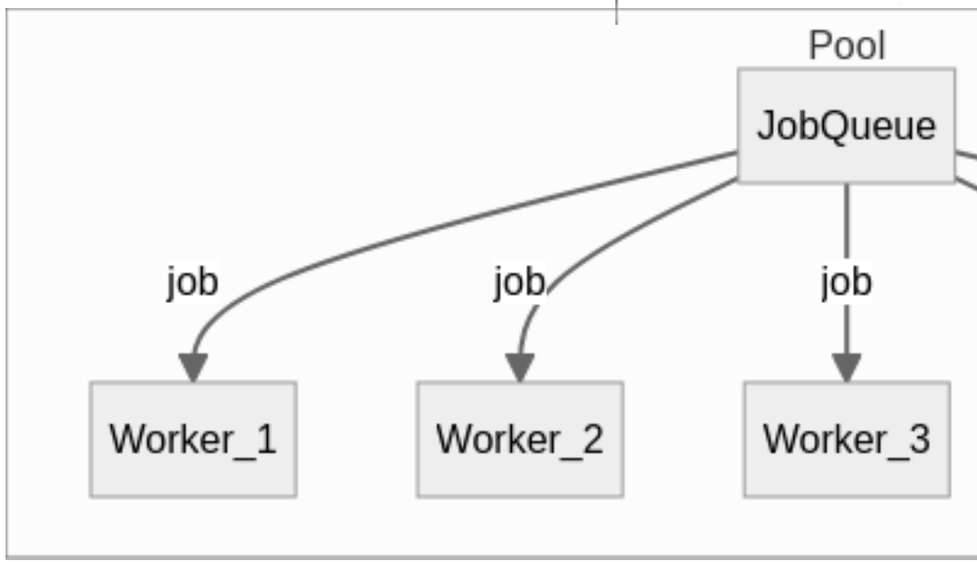
Llenando de tareas un pool

Para asignar las tareas al pool creado se debe utilizar alguno de los siguientes métodos:

- map(func, iterable[, chunksize]): toma func y permite su ejecucion entre los procesos, con los argumentos dados por el iterable. Esta función bloquea al programa emisor hasta el fin de su ejecución
- map_async(func, iterable[, chunksize]): igual a map, pero no bloquea.
- starmap/ starmap_async: igual a los anteriores pero los iterables pueden conener iterables





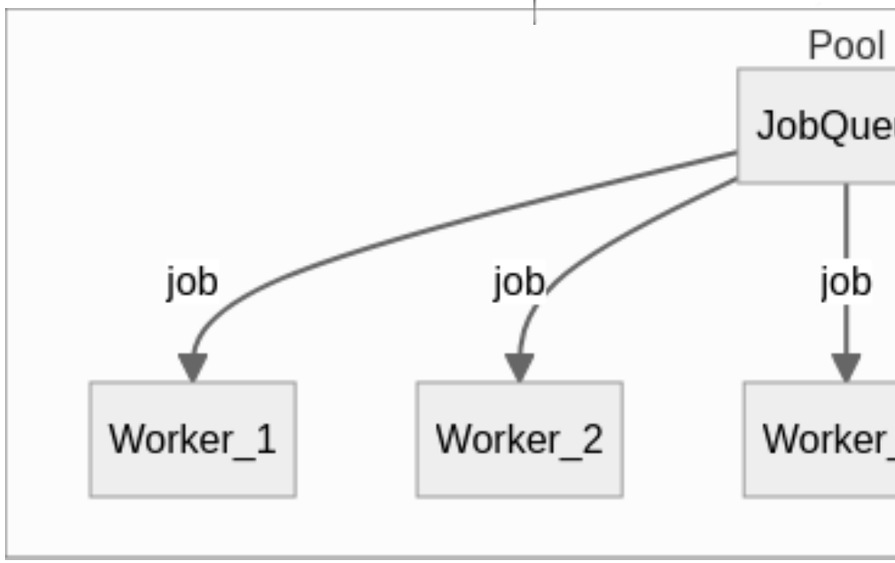


Herramientas de métodos async

Como los métodos map_async y starmap_async no son bloqueantes, se tiene un set de opciones adicionales para el procesamiento de los pools creados



- get([timeout]): Retorna los resultados del pool en forma de lista. Funcion bloqueante
- wait([timeout]): Espera hasta que los procesos estén listos. Función bloqueante
- ready(): Retorna True or False dependiendo si los procesos han terminado de ejecutarse
- succesful(): Retorna True or False dependiendo si los procesos han arrojado algún error. En caso de no terminar se arroja una excepción ValueError.



UNIVERSIDAD CATOLICA DEL MAULE

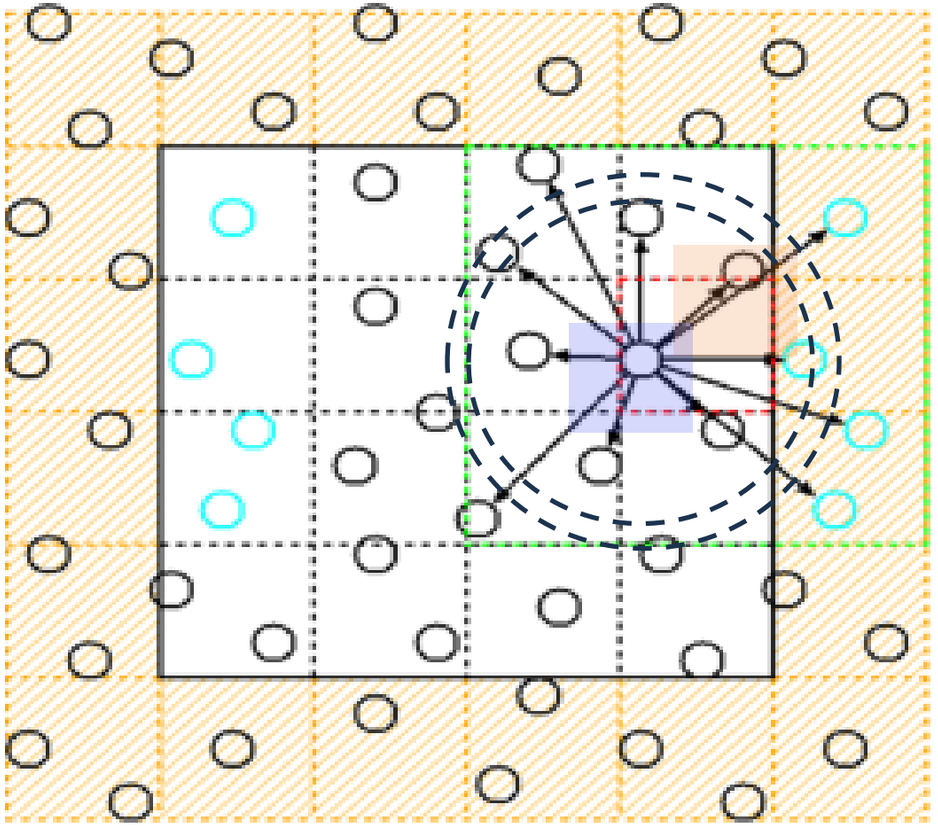
Problema práctico: Función de distribución radial

Función de distribución radial

El cálculo de distribución radial requiere conocer la densidad del entorno a medida que se llega al entorno local de la estructura. Esta tiene un radio de corte hasta el cual se considera el cálculo

- Se cuenta la cantidad de elementos en un conjunto de distancias. Prácticamente es un histograma de coordinaciones para distintas distancias

- Luego se normaliza la candidad obtenida por el volumen de cada casaron (V_(R+dr)-V_R), la candidad de elementos sobre el que se hizo el promedio (N) y la densidad media del sistema (N/V)





Actividad

Calculando la función de distribución radial

UNIVERSIDAD CATOLICA DEL MAULE

Dado el problema de los radios de corte. Paralelizar un algoritmo que entregue la función de distribución radial para el sistema previamente definido en la plataforma

- Leer la estructura dada en la plataforma
- Adapte las funciones del calculo de listas de vecinos para que puedan trabajar incluir bordes periódicos.
- Utilice el método pool para distribuir el trabajo en paralelo del calculo de listas de vecinos en 10 radios de corte diferentes desde 3 a 15.

Notar que el radio arbitrario no le permite definir la cantidad de tareas que se van a distribuir a priori.

- La rutina debe ser capaz de tabajar sobre 4 procesos independiente del radio de corte utilizado.
- Compare la eficiencia del código para los distintos radios de corte utilizados

```
def fun_paralela(iid):
  ...

if __name__ == '__main__':
    p_num = 8
    arr = mp.Array('i', [0
                         lock=Fal
    # Creando lista de proc
    procs = ???

    # Corriendo tareas
    for p in procs:
        p.start()

    # Esperar a que termine
    for p in procs:
        p.join()
```